

Critical Reproduction of a Phishing Website Detection Tutorial

Final Project - Data Science in Cybersecurity

Layan Awawde

July 2026

Selected source: Nicolas Papernot, “Detecting phishing websites using a decision tree” (2016)

Project repository: <https://github.com/layanawawde/phishing-detection-critical-evaluation>

Executive Summary

This project critically evaluates Nicolas Papernot's 2016 blog tutorial on phishing website detection with a decision tree. The source claims approximately 90.5% accuracy using 30 handcrafted website indicators. The current repository file contains 11,055 rows, even though the tutorial states that the dataset contains 2,456 websites. Because the original code trains on the first 2,000 rows and tests on all remaining rows, the current execution tests 9,055 rows rather than the documented 456.

The numerical claim can be reproduced only under that unintended current-file behavior: across 50 deterministic seeds, the mean accuracy is 0.9059. When the experiment is restricted to the first 2,456 rows, leaving the stated 456-row test set, the mean accuracy is about 0.9643. The repository also contains an output-format inconsistency: the script multiplies accuracy by 100 while the README displays a decimal result near 0.906.

A more serious issue is target semantics. The tutorial prose says that +1 denotes phishing and -1 denotes legitimate, while the established UCI convention for this dataset is +1 legitimate and -1 phishing. Accuracy is unchanged by this reversal, but phishing-specific precision, recall, F-scores, false positives, and false negatives are interpreted incorrectly unless the mapping is corrected. This project therefore defines phishing as the positive class when Result = -1.

The raw dataset contains 5,206 exact duplicate repetitions, 5,270 repeated feature-vector occurrences, and 64 unique feature vectors that appear with contradictory labels. The primary experiment retains one row per unique, consistently labeled vector and excludes contradictory groups, leaving 5,721 observations. Logistic Regression, a controlled Decision Tree, and Random Forest are evaluated with a stratified 80/20 holdout and stratified five-fold cross-validation. Random Forest achieves holdout F1 = 0.9695 and mean cross-validated F1 = 0.9606. Removing three highly redundant features produces similar or slightly better holdout performance in this split.

The final conclusion is that the tutorial is useful as an introductory demonstration but does not support production deployment. It lacks consistent documentation, a complete raw-data and feature-extraction pipeline, duplicate-aware evaluation, temporal or campaign identifiers, external validation, operational threshold selection, and robustness analysis against evolving or adversarial phishing behavior.

Primary recommendation: do not deploy the tutorial model unchanged; use it only as a baseline for a redesigned, time-aware and externally validated study.

1. Summary of the Source

1.1 Problem and importance

The selected source addresses binary classification of phishing versus legitimate websites. Phishing websites imitate trusted services to steal credentials, financial details, or other sensitive information. Automatic detection is important because blocklists can miss newly created pages, while manual analysis does not scale.

1.2 Proposed solution

The tutorial trains a scikit-learn DecisionTreeClassifier on 30 pre-engineered indicators. Examples include URL length, use of an IP address, prefix/suffix characters, SSL state, request-URL behavior, anchor behavior, domain age, web traffic, page rank, and JavaScript-related indicators. The repository also includes a Logistic Regression alternative.

1.3 Dataset

The source attributes the data to the UCI Phishing Websites dataset. UCI currently documents 11,055 instances, 30 integer predictors, no missing values, and a CC BY 4.0 license [3]. The source repository includes a headerless dataset.csv file and a short feature list, but not the raw URLs or the complete extraction tool.

1.4 Main claims

- The decision tree achieves approximately 90.5% accuracy.
- The dataset contains 2,456 websites, with 2,000 used for training and 456 for testing.
- The source prose states that +1 is phishing and -1 is legitimate.
- The decision tree performs better than the included Logistic Regression example.

2. Critical Evaluation

The first claim is reproducible only under the current repository file and the code's actual slicing behavior. Table 1 shows that the stated 456-row test does not reproduce 90.5%.

Scenario	Mean accuracy	SD	Minimum	Maximum
Current file: 2,000 train / 9,055 test	0.9059	0.0017	0.9014	0.9092
Documented size: 2,000 train / 456 test	0.9643	0.0019	0.9605	0.9693

Table 1. Reproduction of the source split across 50 decision-tree seeds.

2.1 Documentation and implementation inconsistencies

Issue	Source statement	Observed behavior	Severity
Dataset size	README: 2,456 rows	Current CSV: 11,055 rows	Major
Test size	README: 456 rows	Code uses every row after 2,000; current test = 9,055	Major
Target mapping	README: +1 phishing, -1 legitimate	UCI convention: +1 legitimate, -1 phishing	Major
Output scale	README sample: about 0.906	Current code computes 100 × accuracy	Moderate

Randomness	No seed discussed	DecisionTreeClassifier has tie-breaking variability	Moderate
Environment	Install latest scikit-learn	Python 2 print syntax; no pinned versions	Moderate

Table 2. Source inconsistencies identified during reproduction.

2.2 Are the conclusions justified?

The claim that a decision tree can fit this benchmark with high accuracy is supported. A broader claim that the method is reliable for operational phishing detection is not supported. The evaluation uses one row-order split, accuracy only, no uncertainty estimate, no duplicate analysis, no temporal test, no external dataset, and no campaign-aware grouping. The current file is old and heavily processed; static IID performance cannot establish future or adversarial robustness.

2.3 Label semantics and cybersecurity consequences

The label reversal is especially important. Reversing the positive class leaves accuracy unchanged but swaps the operational meanings of false positives and false negatives. In a phishing system, a false negative means that a phishing page is treated as legitimate, which may expose user credentials. This report and notebook consistently define phishing as positive class 1 after converting UCI Result = -1.

This is the most important correction in the revised project.

3. Feature Engineering Analysis

3.1 Existing engineered features

The source does not model raw URLs. It consumes 30 rule-based indicators already encoded as -1, 0, or 1. Substantial upstream feature engineering has therefore occurred, but the full extraction pipeline is hidden. The repository does not provide collection timestamps, raw domains, raw HTML, certificates, DNS histories, or the program that computes every indicator.

3.2 Transformations in this reproduction

- Target correction: Result = -1 is converted to phishing = 1; Result = +1 becomes legitimate = 0.
- Duplicate control: one row is retained per unique, consistently labeled feature vector; contradictory vectors are excluded from the primary experiment.
- Standardization: StandardScaler is applied only inside the Logistic Regression pipeline. Tree models are not scaled.
- No one-hot encoding: many values have an intended ordered risk interpretation. One-hot encoding is possible, but would increase dimensionality and discard the source's ordinal convention.
- No PCA: only 30 predictors are present, and linear combinations would reduce cybersecurity interpretability.
- No resampling: the class distribution is mild enough that stratification and class-sensitive metrics are preferable to synthetic interpolation of ternary rules.

3.3 Redundancy

Spearman correlation is used because the predictors are discrete ordered indicators and normal continuous assumptions are not appropriate. Three training-set pairs exceed $|\rho| = 0.80$. Within each pair, the lower-mutual-information feature is removed in a sensitivity experiment: double_slash_redirecting, popUpWidnow, and port. The reduced 27-feature forest performs similarly, indicating that redundancy can be reduced without degrading this holdout.

Feature set	Accuracy	Precision	Recall	F1	MCC	ROC-AUC	PR-AUC
All 30 features	0.9686	0.9711	0.9679	0.9695	0.9371	0.9963	0.9967
Reduced (27 features)	0.9738	0.9746	0.9746	0.9746	0.9475	0.9964	0.9967

Table 3. Random Forest redundancy sensitivity on the primary holdout.

3.4 Additional features

Potential improvements include lexical character n-grams, URL entropy, registrar and hosting-AS reputation, certificate age and issuer, DNS change history, redirect-chain depth, brand-visual similarity, form destinations, JavaScript obfuscation, campaign clustering, and time since first observation. These features may improve performance but increase collection cost, latency, privacy risk, and attacker awareness. Their value should be tested with recent time-stamped data rather than assumed.

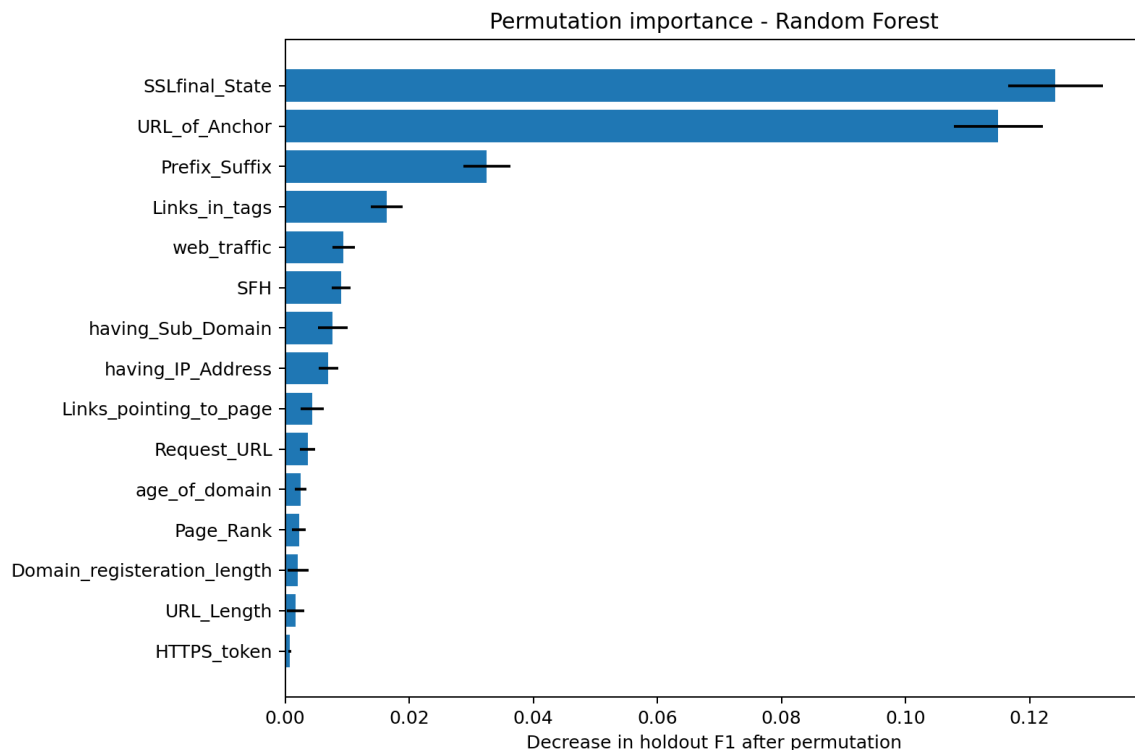


Figure 1. Holdout permutation importance for the best model.

Feature	Mean F1 decrease	SD
SSLfinal_State	0.1242	0.0076
URL_of_Anchor	0.1150	0.0072
Prefix_Suffix	0.0325	0.0038

Links_in_tags	0.0164	0.0026
web_traffic	0.0094	0.0018
SFH	0.0090	0.0015
having_Sub_Domain	0.0077	0.0024
having_IP_Address	0.0070	0.0016
Links_pointing_to_page	0.0044	0.0018
Request_URL	0.0036	0.0012

Table 4. Ten most important predictors by permutation decrease in holdout F1.

4. Reproducibility Analysis

4.1 What is reproducible

- The current CSV and feature names are publicly available.
- The original row slicing and decision-tree training can be reimplemented.
- The revised notebook executes end-to-end with fixed seeds, documented dependencies, generated figures, and exported result tables.
- The public repository includes a PDF report, executed notebook, dataset, README, helper modules, figures, and exact verified package versions.

4.2 What is not reproducible

- The raw website collection process cannot be reconstructed from the tutorial repository.
- The exact feature-extraction pipeline is absent.
- The origin of the 2,456-row statement versus the current 11,055-row file is not documented.
- The source provides no timestamp, URL, domain, campaign, or organization identifiers.
- The source uses Python 2 print syntax and no pinned environment.
- The original code does not set a random seed, so tree tie-breaking can vary.

Overall reproducibility is partial: the supplied table can be modeled, but the full data-generation experiment cannot be independently rebuilt.

4.3 Execution instructions

From the repository root, install requirements and execute the notebook with:

```
python -m pip install -r requirements.txt
python -m jupyter nbconvert --to notebook --execute notebooks/phishing_detection_reproduction.ipynb --output
phishing_detection_reproduction_verified.ipynb --ExecutePreprocessor.timeout=900
```

5. Experimental Results

5.1 Data loading, inspection, and EDA

The raw file contains 11,055 rows, 30 predictors, no missing values, no constant predictors, and only permitted indicator values. Since the predictors are bounded categorical/ordinal codes, continuous outlier methods such as z-scores and IQR fences are not meaningful. The relevant anomalies are invalid codes, rare categories, near-constant features, duplicates, and contradictory labels.

Dataset	Rows	Phishing rows	Phishing prevalence
Raw	11055	4898	44.31%
Unique consistent feature vectors	5721	2955	51.65%

Table 5. Data cleaning and prevalence.

At 44.31%, phishing prevalence is far above many deployment streams. The source does not discuss this mismatch, which can understate alert-volume problems. No oversampling is used because imbalance is mild and synthetic ternary vectors are difficult to justify.

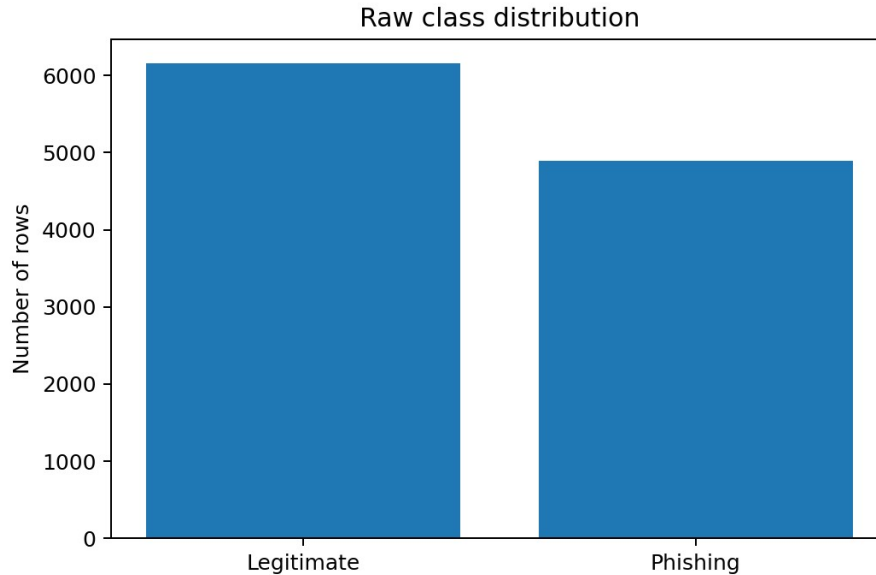


Figure 2. Raw class distribution after correcting the UCI label meaning.

5.2 Columns, index, time, and concentration

The default RangeIndex has no semantic meaning. There are no URL, domain, campaign, or timestamp identifiers, so campaign-aware grouping and temporal validation are impossible. This is a substantive limitation because phishing behavior changes over time. Several fields are near-constant; RightClick takes its modal value in 95.69% of raw rows.

Feature	Unique values	Mode	Mode coverage
RightClick	2	1	95.69%
Iframe	2	1	90.85%
Redirect	2	0	88.43%
on_mouseover	2	1	88.10%
double_slash_redirecting	2	1	87.07%
Shortining_Service	2	1	86.94%
Prefix_Suffix	2	-1	86.75%
port	2	1	86.41%
Google_Index	2	1	86.08%
Statistical_report	2	1	85.98%

Table 6. Most concentrated predictors.

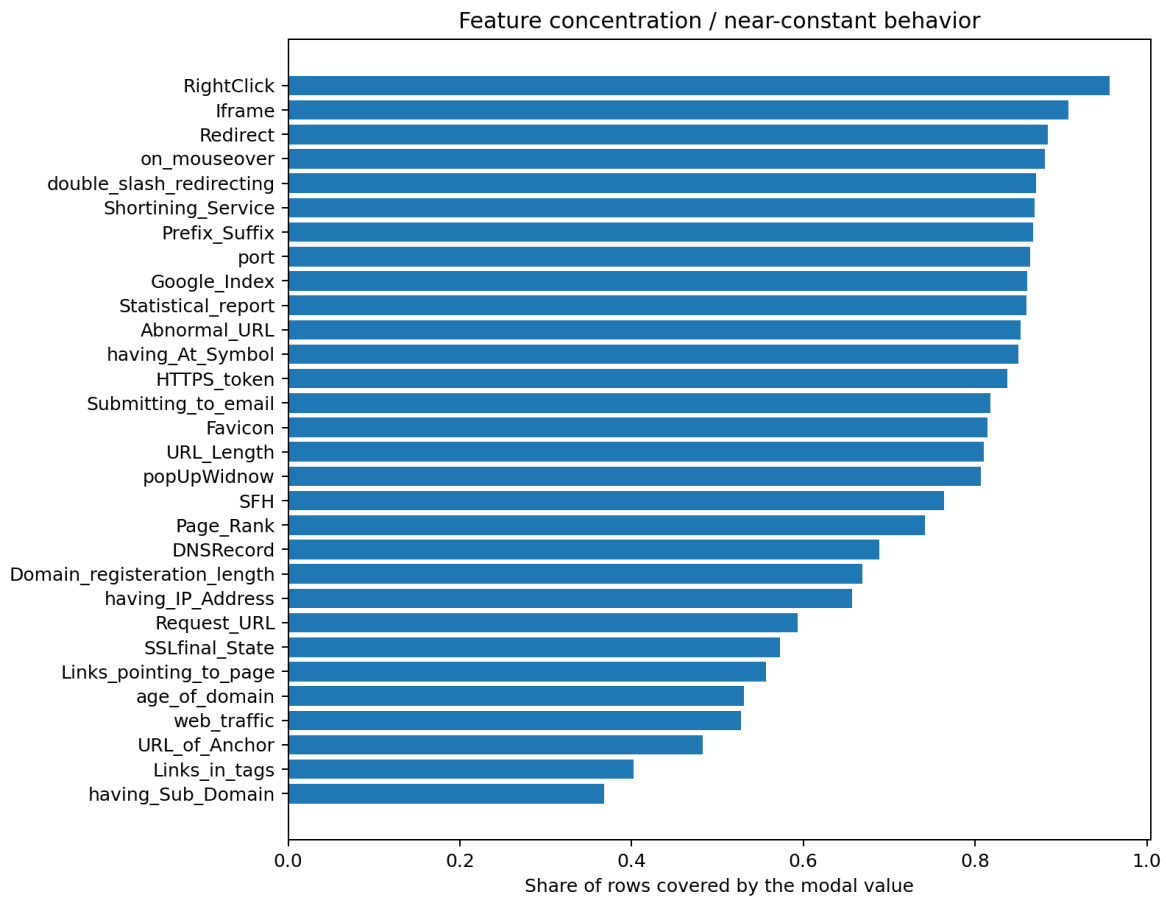


Figure 3. Modal coverage of all predictors.

5.3 Correlation and practical significance

Spearman correlation measures monotonic rank association and is suitable for ordered discrete indicators. Mathematically, $\rho_s = \text{corr}(\text{rank}(X), \text{rank}(Y))$. With more than 5,000 observations, many weak effects have very small p-values, so practical interpretation is based primarily on $|\rho_s|$. SSLfinal_State and URL_of_Anchor have the strongest associations with phishing. The sign reflects the UCI coding direction; it is not a causal effect. Spearman does not capture every nonlinear relationship and tied ranks are common in this ternary dataset.

Feature	Spearman rho	p-value	Practical strength
SSLfinal_State	-0.724	< 1e-300	strong
URL_of_Anchor	-0.699	< 1e-300	strong
Prefix_Suffix	-0.386	6.34e-203	moderate
having_Sub_Domain	-0.310	2.62e-127	moderate
web_traffic	-0.303	1.27e-121	moderate
SFH	-0.282	3.72e-105	weak
Links_in_tags	-0.252	8.88e-84	weak
Request_URL	-0.243	1.85e-77	weak
Domain_registration_length	0.224	9.74e-66	weak
Google_Index	-0.120	1.02e-19	weak

Table 7. Strongest Spearman associations with the corrected phishing target.

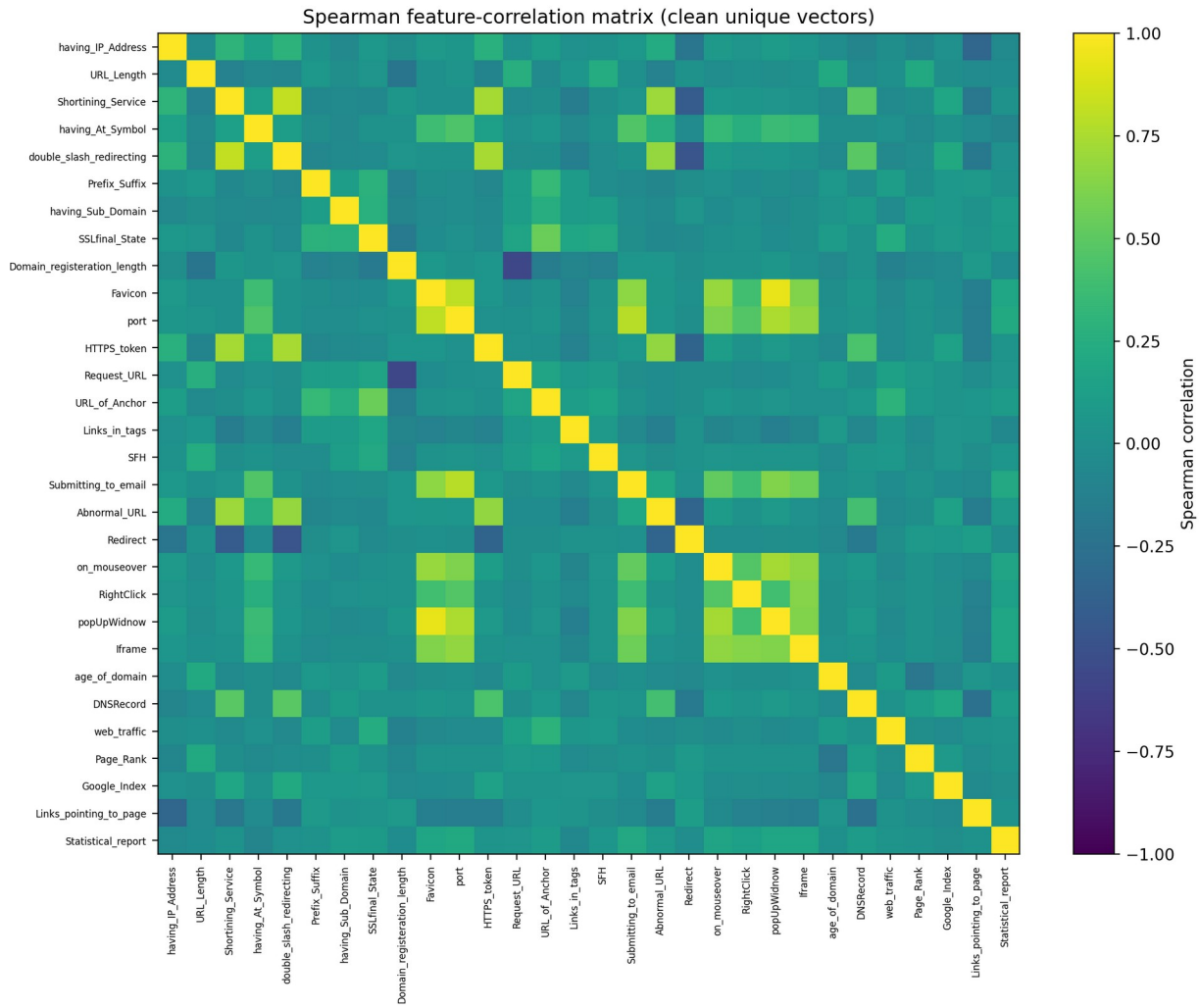


Figure 4. Spearman correlation matrix on unique consistent feature vectors.

5.4 Models and validation design

The primary cleaned dataset contains 5,721 unique, consistently labeled feature vectors. A stratified 80/20 split with `random_state = 42` is used, followed by stratified five-fold cross-validation. The models are Logistic Regression with `StandardScaler`, a Decision Tree with `max_depth = 8` and `min_samples_leaf = 5`, and a 200-tree Random Forest with `min_samples_leaf = 2`.

5.5 Evaluation metrics

Phishing is the positive class. Accuracy is retained for comparison with the tutorial, but the analysis emphasizes Precision, Recall, F1, F2, MCC, Balanced Accuracy, ROC-AUC, PR-AUC, and the confusion matrix.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

$$\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$F\beta = (1 + \beta^2) \times \text{Precision} \times \text{Recall} / (\beta^2 \times \text{Precision} + \text{Recall})$$

$$\text{MCC} = (\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) / \sqrt{[(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})]}$$

$$\text{Balanced Accuracy} = (\text{Recall} + \text{Specificity}) / 2$$

$$\text{ROC-AUC} = P(\text{score}(\text{phishing}) > \text{score}(\text{legitimate})), \text{ with half credit for ties}$$

$$\text{PR-AUC} = \int_0^1 \text{Precision}(r) \, dr, \text{ where } r \text{ denotes Recall}$$

F2 uses $\beta = 2$ and gives recall greater weight because missed phishing pages are usually costlier than additional investigations. ROC-AUC measures ranking across all decision thresholds; PR-AUC focuses on positive-class retrieval and is especially informative when phishing is rare. MCC summarizes all four confusion-matrix cells. Regression metrics are omitted because the task is classification.

5.6 Holdout results

Model	Accuracy	Balanced Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC	PR-AUC
Random Forest	0.9686	0.9686	0.9711	0.9679	0.9695	0.9685	0.9371	0.9963	0.9967
Logistic Regression	0.9459	0.9461	0.9552	0.9391	0.9471	0.9423	0.8918	0.9854	0.9866
Decision Tree	0.9459	0.9464	0.9649	0.9289	0.9466	0.9359	0.8924	0.9887	0.9880

Table 8. Primary cleaned holdout results; phishing is the positive class.

Random Forest gives the strongest holdout F1, MCC, ROC-AUC, and PR-AUC. The confusion matrix contains 17 false positives and 19 false negatives at threshold 0.5.

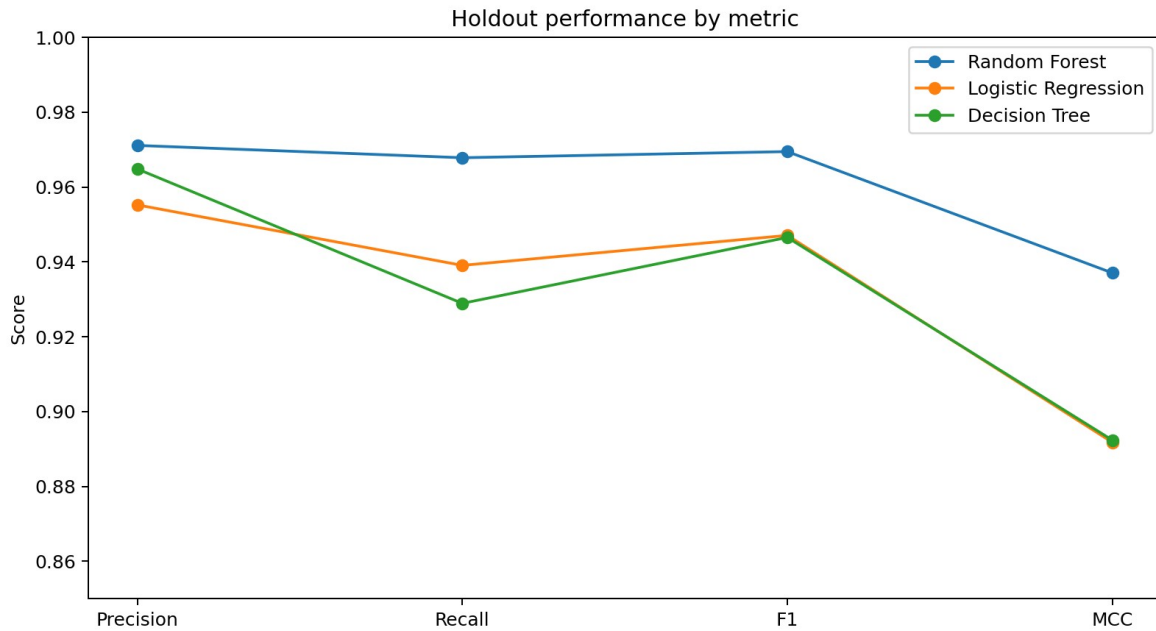


Figure 5. Holdout model comparison.

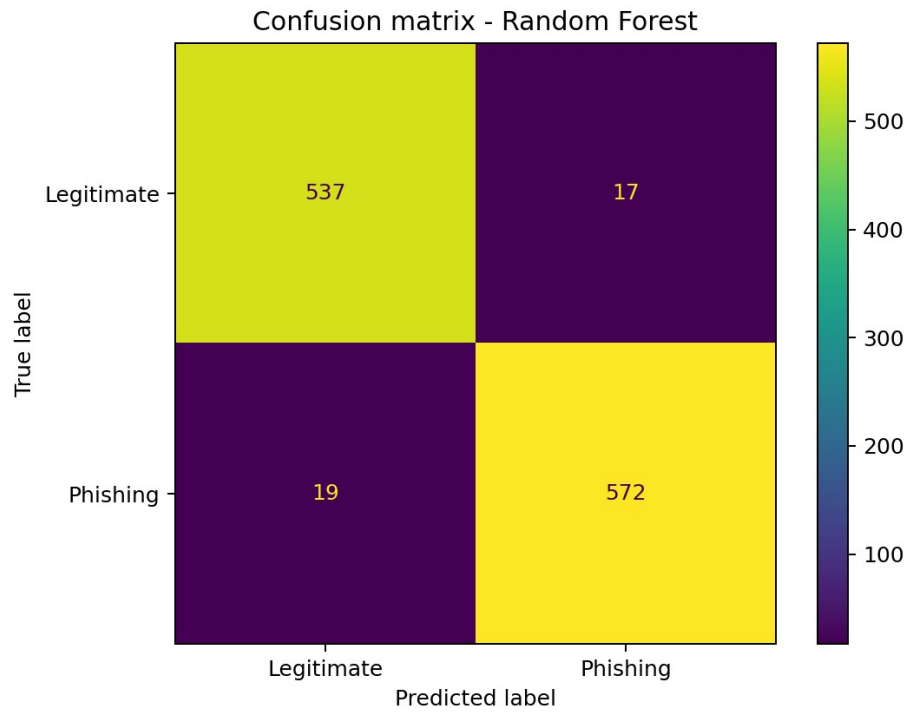


Figure 6. Random Forest confusion matrix on the primary holdout.

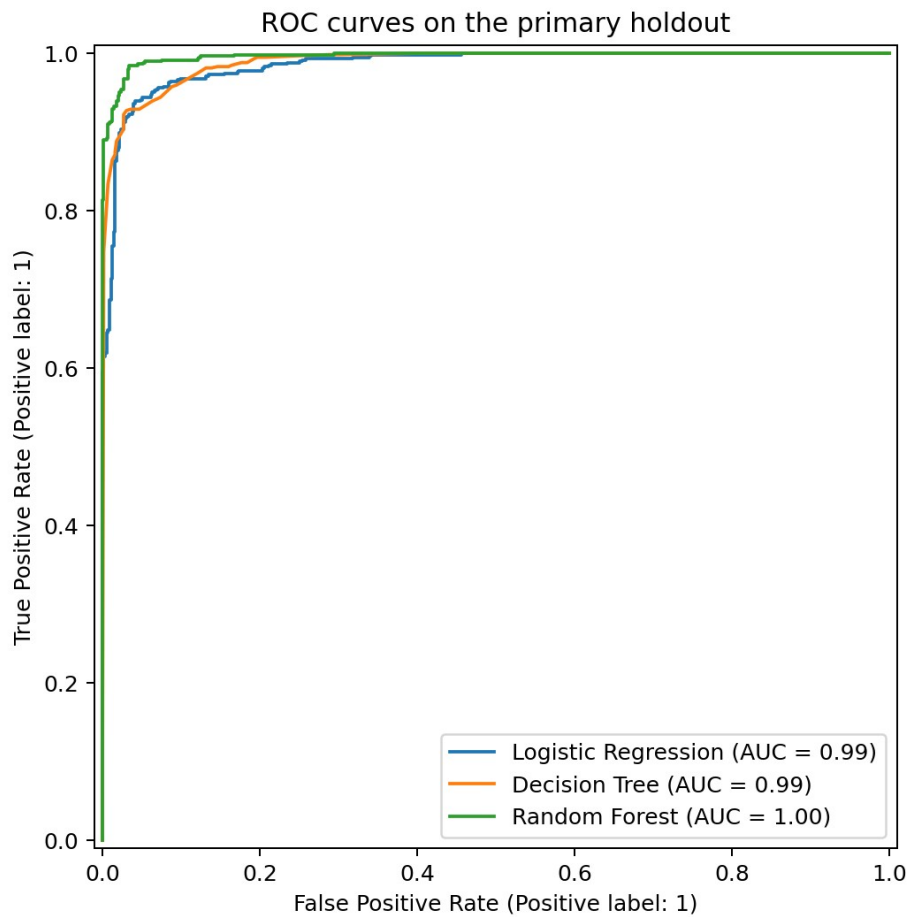


Figure 7. ROC curves on the primary holdout.

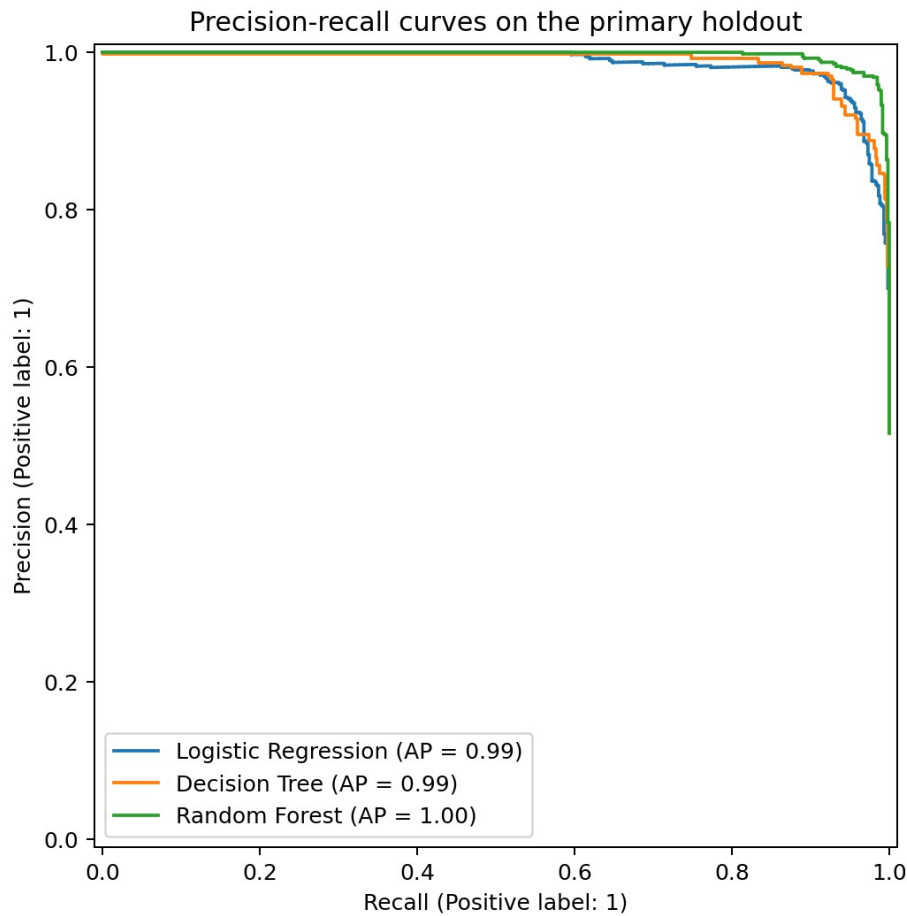


Figure 8. Precision-recall curves on the primary holdout.

5.7 Cross-validation results

Model	Accuracy mean	Accuracy SD	Precision mean	Recall mean	F1 mean	F1 SD	ROC-AUC mean	PR-AUC mean
Logistic Regression	0.9287	0.0057	0.9367	0.9245	0.9305	0.0055	0.9805	0.9827
Decision Tree	0.9360	0.0055	0.9462	0.9289	0.9375	0.0058	0.9818	0.9811
Random Forest	0.9596	0.0070	0.9677	0.9536	0.9606	0.0070	0.9934	0.9941

Table 9. Stratified five-fold cross-validation mean and standard deviation.

The Random Forest mean F1 is 0.9606 with standard deviation 0.0070. Cross-validation reduces dependence on one split but cannot compensate for missing timestamps, campaign IDs, or an external test set.

5.8 Error analysis

A false positive blocks or escalates a legitimate site, creating user friction and analyst fatigue. A false negative allows a phishing site to appear legitimate and may expose credentials or payment information. The 19 false negatives tend to have apparently legitimate values on the strongest predictors, especially `SSLfinal_State` and `URL_of_Anchor`. The 17 false positives show the inverse pattern. This indicates that errors are concentrated in feature vectors whose strongest rules conflict with their labels.

The dataset does not include raw URLs, so failures cannot be reviewed as webpages. This limits root-cause analysis and prevents checking whether labels or feature extraction are wrong. Table 10 gives concrete high-confidence mistakes as engineered feature vectors; it should not be mistaken for webpage-level forensic evidence.

Error type	Phishing probability	SSL state	Anchor	Prefix/suffix	Web traffic	Sub-domain	True phishing	Predicted phishing
False Negative	0.0963	1	0	-1	1	0	1	0
False Negative	0.1773	1	0	-1	0	1	1	0
False Negative	0.2258	1	0	-1	1	0	1	0
False Positive	0.8220	-1	0	-1	0	0	0	1
False Positive	0.7575	-1	0	-1	-1	0	0	1
False Positive	0.7570	-1	0	-1	1	-1	0	1

Table 10. Concrete high-confidence Random Forest error examples at threshold 0.5.

Threshold	Precision	Recall	F1	F2	FP	FN
0.35	0.9511	0.9882	0.9693	0.9805	30	7
0.40	0.9636	0.9848	0.9741	0.9805	22	9
0.50	0.9711	0.9679	0.9695	0.9685	17	19
0.60	0.9806	0.9391	0.9594	0.9471	11	36
0.70	0.9872	0.9137	0.9490	0.9275	7	51

Table 11. Random Forest threshold trade-off.

At threshold 0.40, false negatives decrease from 19 to 9 and recall rises to 0.9848, while false positives increase from 17 to 22. At threshold 0.35, false negatives fall to 7 but false positives rise to 30. The deployment threshold should therefore reflect organization-specific incident costs and analyst capacity.

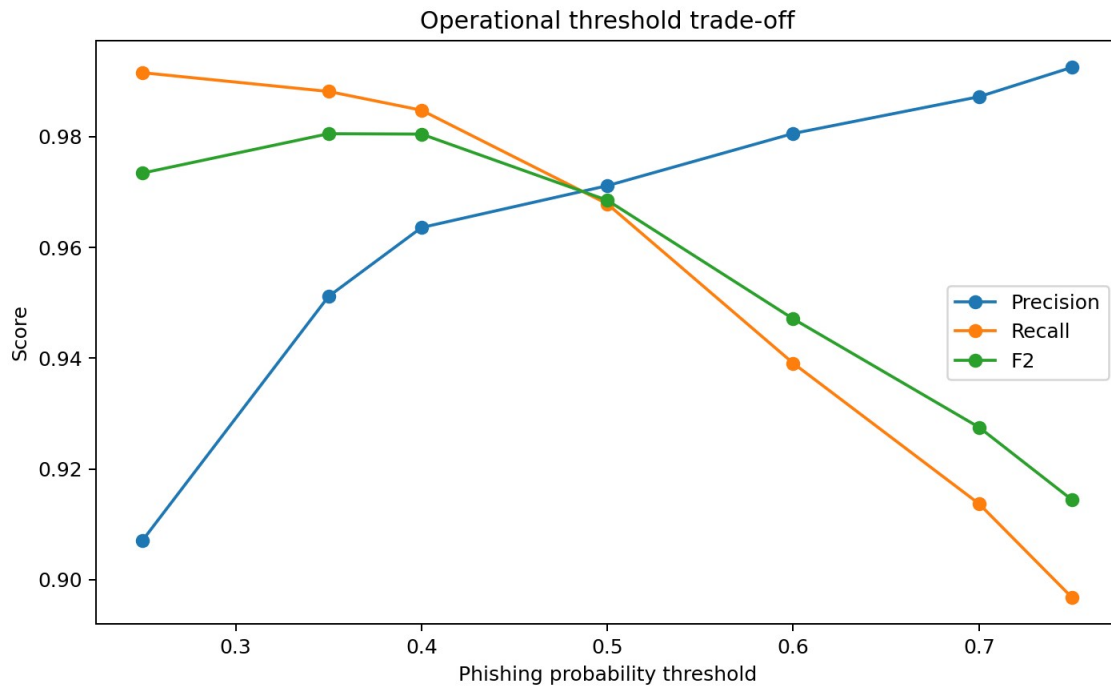


Figure 9. Precision, recall, and F2 across phishing probability thresholds.

6. Conclusions

6.1 Key findings

1. The 90.5% accuracy claim is approximately reproduced only with the current file's unintended 9,055-row test set.
2. The documented 456-row test produces about 96%, not 90.5%, on the current file.
3. The tutorial prose reverses the UCI target meanings, which invalidates phishing-specific error interpretation unless corrected.
4. The dataset contains extensive duplicate structure and 64 contradictory-label feature-vector groups.
5. Random Forest performs best on the cleaned benchmark, but static IID results do not establish real-world generalization.
6. Removing three highly redundant variables does not harm the holdout result, improving simplicity.
7. The absence of timestamps, campaigns, raw URLs, and a complete feature extractor is the dominant reproducibility and validity limitation.

6.2 Strengths and weaknesses of the proposed solution

Strengths include a simple and interpretable baseline, publicly available tabular data, inexpensive training, and features with direct security meaning. Weaknesses include stale and undocumented data provenance, reversed label prose, hidden preprocessing, duplicate and contradictory observations, one fixed split, accuracy-only evaluation, no temporal validation, and no adversarial analysis.

6.3 Future improvements

- Collect recent raw URLs with timestamps, campaign IDs, and source provenance.
- Publish the complete feature-extraction code and data dictionary.
- Use chronological, campaign-aware, and external validation splits.
- Calibrate probabilities and select thresholds using explicit false-positive and false-negative costs.
- Evaluate robustness to attacker-controlled feature manipulation and concept drift.
- Compare feature-rich models with simpler models under latency, maintenance, and explainability constraints.

7. Summing It Up

The project evaluates Nicolas Papernot's Medium tutorial and original GitHub repository on decision-tree phishing detection. The supplied UCI-based dataset currently has 11,055 rows and 30 engineered indicators. The study first reproduced the original row slicing, then corrected the label interpretation, removed duplicate leakage and contradictory feature vectors, performed EDA and redundancy analysis, and compared Logistic Regression, Decision Tree, and Random Forest with a stratified holdout and five-fold cross-validation.

The author's 90.5% numerical claim is supported only in a narrow sense: it appears under the current file and its unintended 9,055-row test, not under the documented 456-row test. The source's

broader methodology is not sufficient because the target mapping is described incorrectly, the dataset size is inconsistent, and the evaluation omits duplicate control, multiple metrics, uncertainty, time, campaigns, and external data.

The most important insight is that a high benchmark accuracy does not by itself demonstrate a useful cybersecurity system. Correct target semantics, data provenance, realistic prevalence, duplicate control, error cost, temporal drift, and attacker adaptation are essential. I recommend this source as an educational starting point, but I do not recommend deploying the tutorial model unchanged on similar operational problems.

Final conclusion: the proposed decision-tree solution is easy to reproduce at the table-modeling level, but the original claims require substantial qualification. The revised experiment supports a stronger and more transparent baseline while also showing why production readiness cannot be inferred from this benchmark.

References

- [1] Papernot, N. (2016). Detecting phishing websites using a decision tree. Medium.
<https://medium.com/@NicolasPapernot/detecting-phishing-websites-using-a-decision-tree-ed069d073723>
- [2] Papernot, N. phishing-detection [GitHub repository]. <https://github.com/npapernot/phishing-detection>
- [3] Mohammad, R., & McCluskey, L. (2012). Phishing Websites [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C51W2X>
- [4] Mohammad, R., Thabtah, F., & McCluskey, L. (2015). Phishing Websites Features. University of Huddersfield Repository. <https://eprints.hud.ac.uk/id/eprint/24330/>
- [5] Mohammad, R. M., Thabtah, F., & McCluskey, L. (2012). An assessment of features related to phishing websites using an automated technique. 2012 International Conference for Internet Technology and Secured Transactions, 492-497. IEEE.
- [6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Appendix A. Repository Deliverables

- PDF report and editable DOCX source.
- Fully executed Python notebook with all figures and tables generated from code.
- Dataset copied from the original tutorial repository, with UCI attribution.
- Helper modules for loading, target correction, duplicate handling, and evaluation.
- README with selected blog link, original GitHub link, dataset source, and execution instructions.
- Results folder containing model metrics, correlation tables, threshold analysis, predictions, and contradiction groups.
- Oral-defense notes summarizing the main methodology and findings.